prettier/plugin-phpの仕組みと、PHP code format

PHPerKaigi 2026 / Track C / fujitani sora

fujitani sora

- toridori inc engineer
- TSKaigiの運営 / 技育CAMP 公式メンター
- Maintenar: rfmt, Yamada UI ...
- @_fs0414
- sorafujitani.me

ここに最近のTipsを記載

```
function
greet( $name,   $greeting="
Hello             " ){
                              echo   $greeting . ", " .  $name;
}
```

```
function greet($name, $greeting = "Hello")
{
    echo $greeting . ", " . $name;
}
```

Prettier とは

- 🔧 コードフォーマッター: コードの「意味」を変えずに「見た目」を統一するツール
- https://github.com/prettier
- 🐘 `@prettier/plugin-php` で PHP にも対応
- https://github.com/prettier/plugin-php

Prettierの仕組み

変換パイプライン

```
テキスト → AST → Doc（中間表現）→ テキスト
```

1. **Parse**: Input: source code, Output: AST
2. **PrintAstToDoc**: Input: AST, Output: DocIR
3. **PrintDocToString**: Input: DocIR, Output: Foamatterd String

Parse

- フォーマッターはまず **Parse** (構文解析)を実行する
- コードに存在したスペースや改行の情報は消える

```
function greet($name) {
    echo $name;
}
```

```
program
└── function (name: "greet")
    ├── arguments: [parameter (name: "name")]
    └── body: block
        └── echo
            └── variable (name: "name")
```

```
{
  kind: "function",
  loc: { start: { ... }, end: { ... } },
  name: "greet",
  arguments: [ ... ],
  body: { kind: "block", ... }
}
```

kind

- prettier がノード固有のswitch文で処理

```
"echo"      → echo $x
"function"  → function f() {}
"if"        → if () {} else {}
"call"      → foo($a)
"class"     → class Foo {}
"array"     → [$a, $b]
"variable"  → $name
"foreach"   → foreach () {}
"assign"    → $a = 1
```

loc

ノードが元コードのどこにあったかを示す位置情報

```
echo $name;    // offset: 0〜11
```

```
{ kind: "echo", loc: { start: { offset: 0 }, end: { offset: 11 } } }
```

loc の用途②: 空行の保持

元コードにあった空行だけは残す → ノード間の offset 差で判定

```
$a = 1;
                    // ← 空行がある
$b = 2;
```

```
{
  kind: "expressionstatement",          // $a = 1;
  loc: { start: { offset: 0 },
         end:   { offset: 7 } }
}

{
  kind: "expressionstatement",          // $b = 2;
  loc: { start: { offset: 9 },

         end:   { offset: 16 } }
}
```

loc の用途①: コメントの配置

コメントは AST の木に入らない → offset で最寄りのノードに紐付ける

```
function greet($name)     // offset: 0〜21
{
    // コメントだよ       // ← AST に含まれない
    echo $name;            // offset: 51〜62
}
```

```
{
  kind: "commentline",
  value: "// コメントだよ",
  loc: { start: { offset: 28 },
    end: { offset: 42 } }
}
```

同じ配列でも出力が2パターンある

```
// パターン A
[$a, $b, $c]

// パターン B
[
    $a,
    $b,
    $c,
]
```

printWidth

- prettier の最も基本的な設定。デフォルトは **80 文字**
- この幅に収まるかどうかで、1行にするか改行するかが決まる

```
printWidth: 80
|<──────────────────── 80文字 ────────────────────>|
```

中身が短ければ1行、長ければ改行

```
[$a, $b, $c]                              // 短い → 1行

[$longVariableName1, $longVariableName2]   // 長い → 改行
→ [
      $longVariableName1,
      $longVariableName2,
  ]
```

printWidth (行幅) に収まるかどうかで判断される

ただし、中身の長さだけでは決まらない

- 外側が改行するかは内側の長さ次第。内側の結果は外側の判断次第。

→ 出力内容を決める段階では、改行の判断を保留する必要がある

```
$x = [$a, $b, $c];          // 残り幅が十分 → 1行

$veryLongName = [$a, $b, $c];  // 残り幅が少ない → 改行するかも
```

解決策: Doc IR で判断を遅延させる

AST → テキストの間に **Doc IR** を挟み、2段階に分離する

- **printAstToDoc**: 「何を出力するか」と「改行してもいい場所」を宣言
- **printDocToString**: 行幅に基づいて改行するかを確定

Doc IR

```
[$a, $b]  →  group([ "[", indent([softline, "$a, $b"]), softline, "]" ])
```

- `group` – 1行に収めようとする単位。収まらなければ中の softline を改行に展開
- `softline` – 改行してもいい場所。1行なら消える
- `indent` – 改行したらインデントする

printDocToString の判断

同じ Doc IR でも行幅で結果が変わる:

```
group([ "[", indent([softline, "$a, $b"]), softline, "]" ])

// printWidth: 80 → group が収まる → softline は消える
[$a, $b]

// printWidth: 10 → 収まらない → softline が改行に展開
[
    $a,
    $b,
]
```

色々な構文で同じ仕組み

```
group([ "User::where(", indent([softline, ...args]), softline, ")" ])

//   → 収まる → flat
$users = User::where('active', true)→orderBy('name')→get();

//   → 収まらない → softline が改行に展開
$users = User::where('active', true)
    →whereNotNull('email_verified_at')
    →orderBy('created_at', 'desc')
    →paginate(20);
```

関数呼び出し・配列・メソッドチェーン、全て同じ group + softline で動いている

Doc IR

```
plugin-php   → Doc IR ┐
plugin-js    → Doc IR ┤
plugin-css   → Doc IR ┼→ printDocToString() → テキスト
plugin-html  → Doc IR ┤
plugin-yaml  → Doc IR ┘
```

全ての言語プラグインが Doc IR を出力するので、printDocToString の実装は1つだけでいい

A prettier printer

- Philip Wadler, "*A prettier printer*" (2003)
- `9b4535e` でこのアルゴリズムが導入された

Prettier plugin system

prettier plugin の 5 つのインターフェース

- **languages** – どの拡張子を担当するか
- **parsers** – テキスト → AST
- **printers** – AST → Doc
- **options** – 言語固有の設定項目
- **defaultOptions** – デフォルト値の上書き

pluginの解決

prettier は言語を知らない。拡張子からプラグインを探す:

```
.php → plugin-php
```

パーサーの解決

`languages` が拡張子とパーサー名を紐付け、`parsers` から実装を取得:

```
".php" → languages.parsers: ["php"] → parsers["php"] (内部で php-parser を呼ぶ)
```

```
  "dependencies": {
    "php-parser": "^3.2.5"
  },
```

オプションの解決

```
prettier デフォルト → plugin 上書き → ユーザー設定
(tabWidth: 2)        (tabWidth: 4)   (.prettierrc)
```

`options` と `defaultOptions` でデフォルト値を上書きできる

PHP x Prettier

PER Coding Style と Prettier の思想

Prettier の設計思想: オプションは増やさない

> *"Prettier has a few options because of history. But we won't add more of them." — docs/option-philosophy.md*

braceStyle は追加

PHP コミュニティには PER Coding Style という強い規約文化がある

```
// per-cs（デフォルト）— PHP の慣習
function greet($name)
{
    echo $name;
}

// 1tbs — JS の慣習
function greet($name) {
    echo $name;
}
```

JS の Prettier はこの種のオプション追加を拒否してきた。 PHP プラグインはコミュニティの規約に合わせて妥協した

PER に従わない部分もある

PER Coding Style 3.0 §5.1: 論理演算子は行頭に置く (MUST)

```
// PER
if (
    $conditionA
    && $conditionB
    && $conditionC
) {

// Prettier: 行末に置く
if (
    $conditionA &&
    $conditionB &&
    $conditionC
) {
```

> *"uses PER as guidance… but does not aim to be fully PER compliant" – plugin-php README*

glayzzle/php-parser

plugin-php は PHP 本体のパーサーではなく、JS で書かれた再実装に依存している

```
"dependencies": {
  "php-parser": "^3.2.5"
}
```

https://github.com/glayzzle/php-parser

- 新しい PHP 構文への対応が遅れがち
- AST の情報が不足・不正確なケースがある
- plugin-php 側でワークアラウンドしている箇所が複数ある

php-parser のバグ例: new 演算子の優先順位

https://github.com/glayzzle/php-parser/issues/1177

```
new Foo::$bar→baz();
```

PHP 本体はこれを `new (Foo::$bar→baz)()` と解釈するが、 glayzzle/php-parser は `new Foo::$bar()→baz()` と同じ AST を返してしまう

→ パーサーが間違った AST を返すと、フォーマッターの出力も意味が変わる

※ この問題は PR #1196 で修正済み (2026年3月 merge)

PHP は HTML と混在する

```
<h1>Hello</h1>
<?php echo $name; ?>
<p>Welcome</p>
```

- `?>` と `<?php` の間は HTML – AST 上は `kind: "inline"` ノードになる
- plugin-php の README にも "mixed PHP and HTML is still considered unstable" と記載
- JS や CSS にはない PHP 特有の複雑さ

まとめ

```
prettier test.php
  │
  │   ".php" → plugin-php を解決
  │   オプション: prettier → plugin-php → .prettierrc
  │
  ▼  parse()            テキスト → AST            ← plugin-php
  ▼  printAstToDoc()    AST → Doc                  ← plugin-php
  ▼  printDocToString() Doc → テキスト             ← prettier コア
  │
  ▼  整形済み PHP コード
```

see you later 👋